Preface and Introduction

Computer Organization and Architecture

Computer Architecture acts as the foundational blueprint of a computer system. It dictates the functional behavior of a computer system from the programmer's perspective, involving aspects such as the instruction set, data formats, and memory addressing. Computer Organization, on the other hand, deals with the structural relationships and the operational units that realize this architectural abstraction. Together, they form the core of understanding how digital logic is combined to build complex processing units capable of executing high-level software.

Logisim-Evolution Simulator

The circuits presented in this manual have been designed, simulated, and verified using **Logisim-Evolution**. Logisim-Evolution is an advanced, educational graphical tool for designing and simulating digital logic circuits. It is an upgraded, actively maintained fork of the original Logisim, featuring a modernized interface, real-world electronic components, FPGA board-level simulation support, and the ability to export circuits directly to VHDL or Verilog. This allows for an intuitive, visual approach to building complex architectural components like adders, multiplexers, ALUs, and memory arrays.

GitHub Repository

For full transparency, reproducibility, and source control, the complete Logisim-Evolution project files ('.circ' files) corresponding to the experiments in this manual have been open-sourced.

You can access, download, and review the simulation files on my GitHub repository at the following link:

Repository URL: https://github.com/OFF-rtk/COAVLAB

Experiment 1: Theory and Design

of a 4-Bit Ripple Carry Adder

1 Introduction to Arithmetic Operations

Arithmetic operations such as addition, subtraction, multiplication, and division are fundamental processes implemented in digital computers using basic logic gates (AND, OR, NOR, NAND, etc.). Among these, addition is the most critical operation. Once binary addition is successfully implemented, it can be leveraged to perform other operations:

- **Multiplication** via repeated addition.
- **Subtraction** by negating one operand (using two's complement arithmetic).
- **Division** via repeated subtraction.

2 Adder Architectures

A **Half Adder** is a simple combinational circuit used to add two single-bit binary numbers. However, to add N-bit binary numbers, we must create a logical circuit cascading multiple **Full Adders**.

Each full adder accepts three inputs: two significant bits and a carry-in (C_{in}) which represents the carry-out (C_{out}) generated by the previous lower-order adder. This specific architecture is known as a **Ripple Carry Adder (RCA)**, because each carry bit sequentially "ripples" through the circuit to the next full adder.

Note: For the least significant bit (LSB), the first full adder may be replaced by a half adder if there is no initial carry-in to the system.

2.1 4-Bit Ripple Carry Adder Block Diagram

Below is the structural flow of a 4-bit Ripple Carry Adder adding two 4-bit words, $A(A_3A_2A_1A_0)$ and $B(B_3B_2B_1B_0)$, with an initial carry-in C_0:

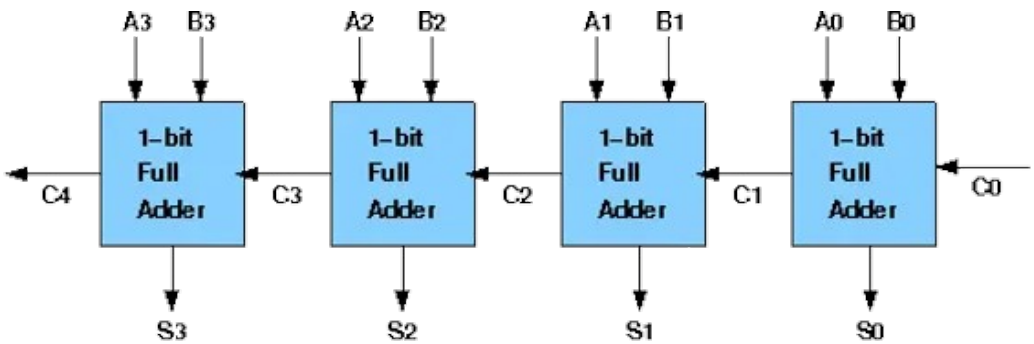

Figure 1: Structural flow of a 4-bit Ripple Carry Adder.

3 Delay and Performance Analysis

The layout of a ripple carry adder is simple and highly structured, which allows for fast design times. However, the RCA is relatively slow in operation. Because each full adder must wait for the carry bit to be calculated and propagated from the previous stage, the delays accumulate.

The gate delay can be calculated by inspecting the full adder circuit. Each full adder typically requires three levels of logic. For instance, in a 32-bit Ripple Carry Adder consisting of 32 full adders, the worst-case critical path delay would be calculated as:

- 31×2 (gate delays for carry propagation across 31 stages)
- $+3$ (gate delays for computing the final sum)
- **Total** = **65 gate delays.**

4 Design Issues and Boolean Expressions

To construct the adder, we must derive the corresponding Boolean expressions.

For a **Half Adder** circuit, the Sum and Carry outputs for inputs A and B are defined as:

- **Sum** = $A \oplus B$
- **Carry** = $A \cdot B$

For a **Full Adder** circuit, the outputs depend on inputs A, B, and C (C_{in}). The unoptimized Sum and Carry-out expressions are formulated using Sum of Products (SOP):

- **Sum** = $A'B'C + A'BC' + AB'C' + ABC$
- **Carry** = $A'BC + AB'C + ABC' + ABC$

While these expressions allow us to design the circuit directly using basic gates, we can check for logically equivalent statements that lead to a more structured and hardware-efficient circuit. With algebraic manipulation, we can optimize these equations:

4.1 Simplifying the Sum

$$
\begin{aligned}
Sum &= A'B'C + A'BC' + AB'C' + ABC \\
Sum &= (A'B' + AB)C + (A'B + AB')C' \\
Sum &= (A \oplus B)'C + (A \oplus B)C' \\
Sum &= A \oplus B \oplus C
\end{aligned}
$$

4.2 Simplifying the Carry

$$
\begin{aligned}
Carry &= A'BC + AB'C + ABC' + ABC \\
Carry &= (A'B + AB')C + AB(C' + C) \\
Carry &= (A \oplus B)C + AB(1) \\
Carry &= AB + (A \oplus B)C
\end{aligned}
$$

By recognizing that $(A \oplus B)$ is computed for the Sum, we can reuse this specific gate output to compute the Carry, significantly reducing the required hardware components and structuring the physical design optimally.

5 Logisim Circuit Implementations

The following figures detail the physical gate-level implementations of the adder components constructed using Logisim Evolution.

5.1 Half Adder Implementation

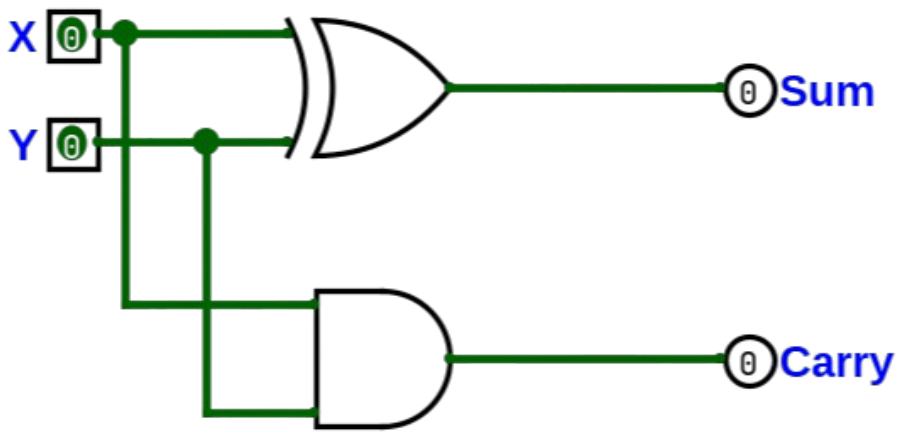

Figure 2: Logic gate implementation of a Half Adder.

5.2 Full Adder Implementation

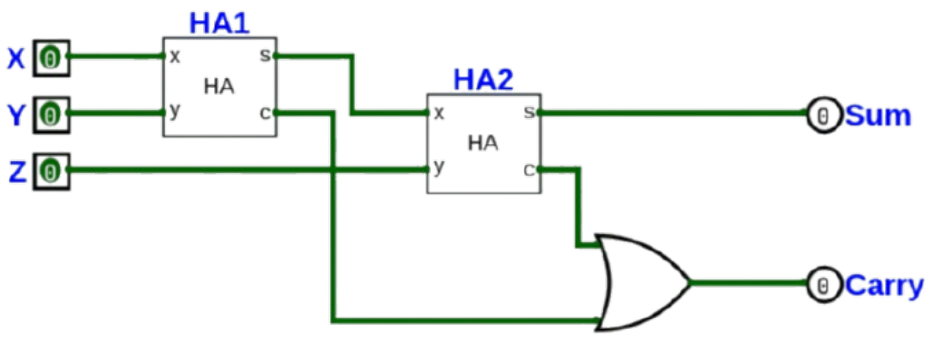

Figure 3: Logic gate implementation of a Full Adder.

5.3 Final 4-Bit RCA Logisim Evolution Diagram

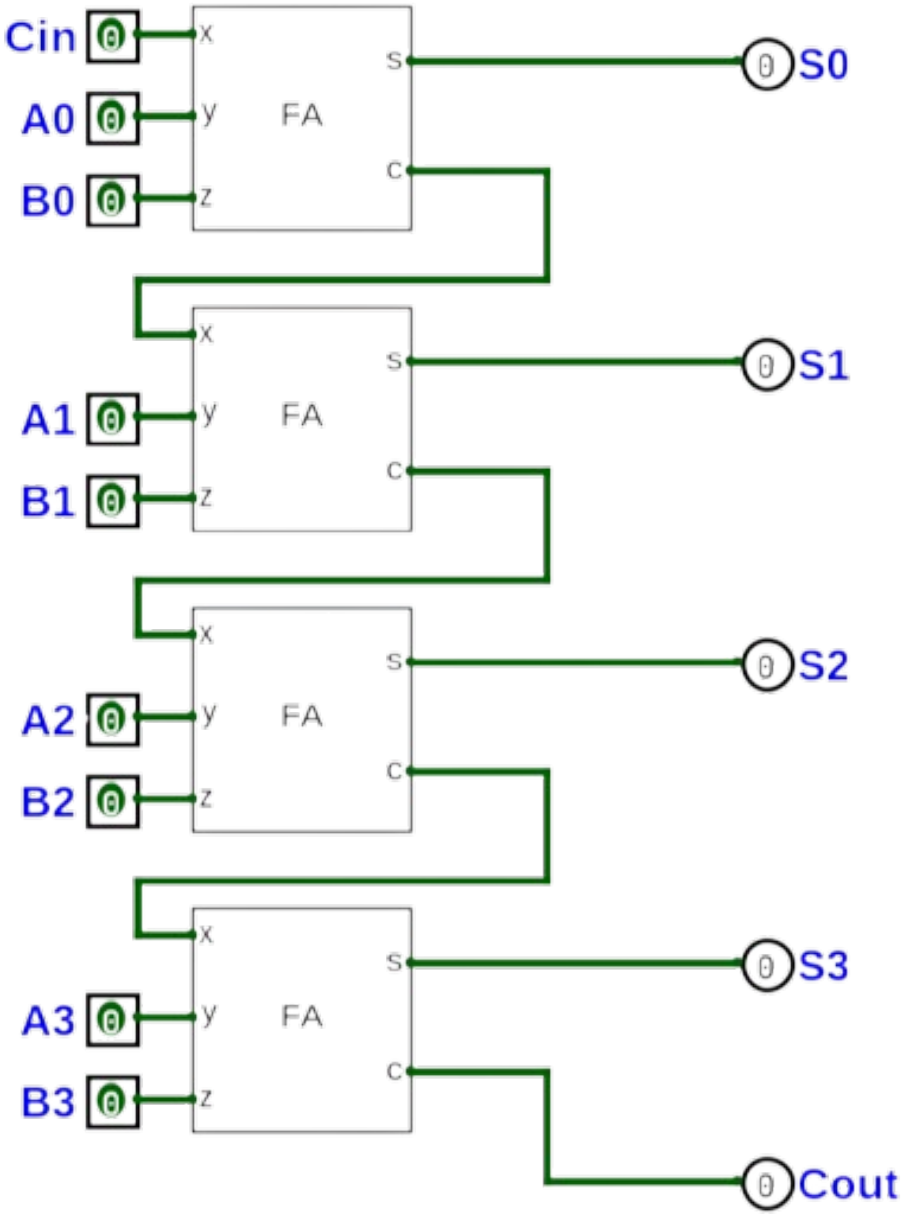

Figure 4: Complete 4-Bit Ripple Carry Adder implemented in Logisim Evolution.

Experiment 2: Simulating Microcontroller Behavior

using Flip-Flops and Logic Gates

1 Aim

To simulate the primitive behavior of a microcontroller's datapath using D flip-flops and logic gates. This experiment focuses on executing a basic logical instruction (AND) synchronously and observing how clock signals govern the flow of data through hardware registers.

2 Theory and Introduction

In digital electronics and systems engineering, the behavior of microcontrollers can be understood by breaking them down into their fundamental logical components. A microcontroller's central processing unit (CPU) relies heavily on two types of logic: **Sequential Logic** (for memory and state) and **Combinational Logic** (for computation).

This experiment isolates and simulates the most basic interaction between a CPU's register file and its Arithmetic Logic Unit (ALU).

2.1 Flip-Flops as Hardware Registers

Flip-flops are bistable multivibrators that serve as the fundamental building blocks of memory elements. In a microcontroller, flip-flops are grouped together to form **Registers** (such as Accumulators or General Purpose Registers) which hold data temporarily during processing.

- **D Flip-Flop (Data):** A D flip-flop captures the input data (D) strictly on the active edge (rising or falling) of a clock signal. This synchronous behavior ensures that data is only updated at discrete time intervals, replicating the instruction cycle of a processor.

2.2 Combinational Logic as an ALU

The Arithmetic Logic Unit (ALU) performs mathematical and bitwise operations. In this simulation, a basic **AND Gate** serves as a primitive, single-instruction ALU. It takes the operands stored in the D flip-flops and computes the logical conjunction.

- **AND Gate:** Outputs a logic 1 only if all input operands are 1.

2.3 The Clock Signal

The clock acts as the heartbeat of the microcontroller system. It synchronizes the state changes of the flip-flops. Data placed at the inputs of the flip-flops will not propagate to the ALU until the clock transitions, demonstrating how a CPU sequences operations.

3 Procedure

The following steps detail the construction of the registered logic circuit in Logisim-Evolution:

1. **Initialize Registers:** Navigate to the *Memory* component library and place two **D Flip-Flops** into the workspace. These act as our two 1-bit input registers.
2. **Configure Inputs:**
   - Add an input pin connected to the D port of the first flip-flop and label it `Input1`.
   - Add an input pin connected to the D port of the second flip-flop and label it `Input2`.
3. **Synchronize the Clock:** Add a **Clock** component and wire it simultaneously to the clock-enable ($>$) pins of both D flip-flops. This ensures both registers load data on the same clock edge.
4. **Implement the ALU Logic:** Place an **AND Gate** in the workspace. Route the output (Q) of the first flip-flop to the top input of the AND gate, and the output (Q) of the second flip-flop to the bottom input.
5. **Observe Output:** Connect a final Output pin to the output node of the AND gate.
6. **Simulation:** Toggle `Input1` and `Input2`. Observe that the final output does not change until the Clock component is manually toggled (simulating a clock tick), proving the synchronous nature of the registers.

4 VLAB Circuit Implementation

4.1 Microcontroller Datapath Simulation Diagram

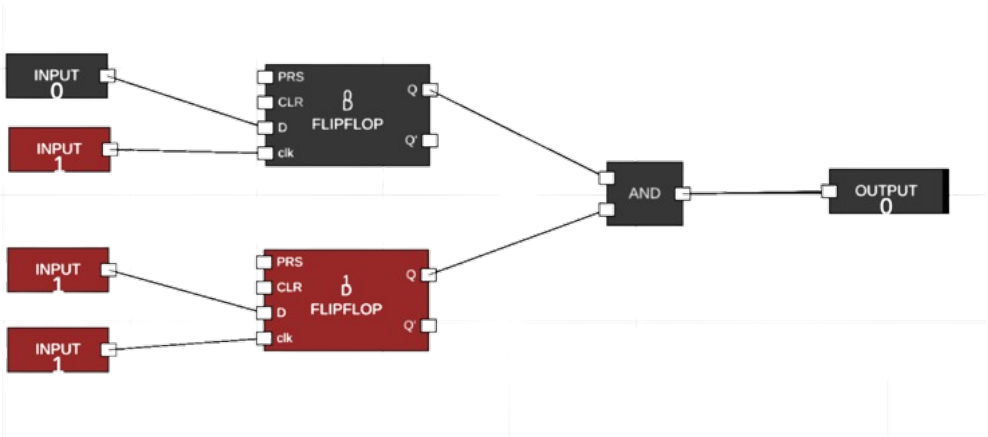

Figure 5: Simulation of a synchronous logical instruction using D Flip-Flops and an AND gate.

Experiment 3: Design of a 4-Bit Carry Lookahead Adder (CLA)

High-Speed Addition via Parallel Carry Generation

1 Aim

To design and simulate a 4-bit Carry Lookahead Adder (CLA) to understand how parallel carry generation reduces propagation delay compared to a Ripple Carry Adder. The experiment focuses on implementing Carry Propagate (P) and Carry Generate (G) functions to compute carries simultaneously.

2 Components Required

The following components and features of **Logisim-Evolution** are utilized in this implementation:

- **Half Adders (HA):** Used as sub-circuits to compute the initial P_i and G_i signals for each bit position.
- **Multi-Input Logic Gates:** High-fan-in AND and OR gates used to implement the expanded carry equations (C_1 through C_4).
- **Logisim Tunnels:** Utilized to manage complex signal routing and avoid wire conflicts, ensuring a clean and readable schematic.
- **Input Pins and Probes:** For data loading and real-time verification of carry bits.

3 Theory

The primary bottleneck in a Ripple Carry Adder is the dependency of each stage on the carry-out of the previous stage. In an n-bit RCA, the carry must pass through $2n$ gate levels. The **Carry Lookahead Adder** eliminates this dependency by calculating all carry bits in parallel based on the input bits.

3.1 Carry Generate and Propagate Functions

For any bit stage i, we define two signals:

- **Generate (G_i):** $G_i = A_i \cdot B_i$. A carry is generated if both inputs are 1, regardless of the input carry.
- **Propagate (P_i):** $P_i = A_i \oplus B_i$. A carry is propagated if one of the inputs is 1.

The Boolean expression for the output carry of a stage is:

$$C_{i+1} = G_i + P_i C_i$$

3.2 Parallel Carry Logic

By substituting the carry equations recursively, we can express every carry bit solely in terms of the initial carry (C_0) and the P/G signals of the current and preceding stages:

$$
\begin{aligned}
C_1 &= G_0 + P_0C_0 \\
C_2 &= G_1 + P_1G_0 + P_1P_0C_0 \\
C_3 &= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \\
C_4 &= G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0
\end{aligned}
$$

Because these equations consist only of Sum-of-Products (SOP) forms, they can be implemented using two-level logic, resulting in a constant delay regardless of the number of bits.

4 Procedure

1. **Pre-computation:** Construct a sub-circuit using a **Half Adder** to output P_i (Sum output) and G_i (Carry output).
2. **Signal Routing:** Use **Tunnels** labeled $P_0 \dots P_3$ and $G_0 \dots G_3$ to distribute these signals to the Lookahead Carry Unit.
3. **Carry Logic Implementation:** Build the combinatorial logic for $C_1, C_2, C_3,$ and C_4 using multi-input AND/OR gates based on the equations in the Theory section.
4. **Sum Generation:** Compute the final sum bits using the relation $S_i = P_i \oplus C_i$.
5. **Verification:** Load the test case $A = 1111$ and $B = 0001$. Observe that $S = 0000$ and $C_{out} = 1$ appear virtually instantaneously across all stages.

5 Functional Truth Table

The following table describes the logic for a single-stage bit i within the Carry Lookahead Adder. By calculating G_i and P_i first, the lookahead carry unit determines C_{i+1} without waiting for a ripple.

Input A_i	**Input B_i**	**G_i (Gen)**	**P_i (Prop)**	**Sum (S_i)**	**Carry Out (C_{i+1})**
0	0	0	0	C_i	0
0	1	0	1	$\overline{C_i}$	C_i
1	0	0	1	$\overline{C_i}$	C_i
1	1	1	0	C_i	1

Table 1: Functional logic of a CLA stage using Generate and Propagate signals.

5.1 Verification Test Case: $1111 + 0001$

Applying the inputs $A = 1111$ and $B = 0001$ with $C_{in} = 0$:

- **Stage 0:** $G_0 = 0, P_0 = 1 \implies C_1 = 0 + 1(0) = 0$.

- **Stage 1:** $G_1 = 0, P_1 = 1 \implies C_2 = 0 + 1(0) = 0$.
- **Stage 2:** $G_2 = 0, P_2 = 1 \implies C_3 = 0 + 1(0) = 0$.
- **Stage 3:** $G_3 = 0, P_3 = 1 \implies C_4 = 0 + 1(0) = 0$.
- **Final Result:** $Sum = 0000$ with $C_{out} = 1$ (generated by the final Lookahead logic).

6 Logisim Circuit Implementation

6.1 4-Bit CLA Gate-Level Schematic

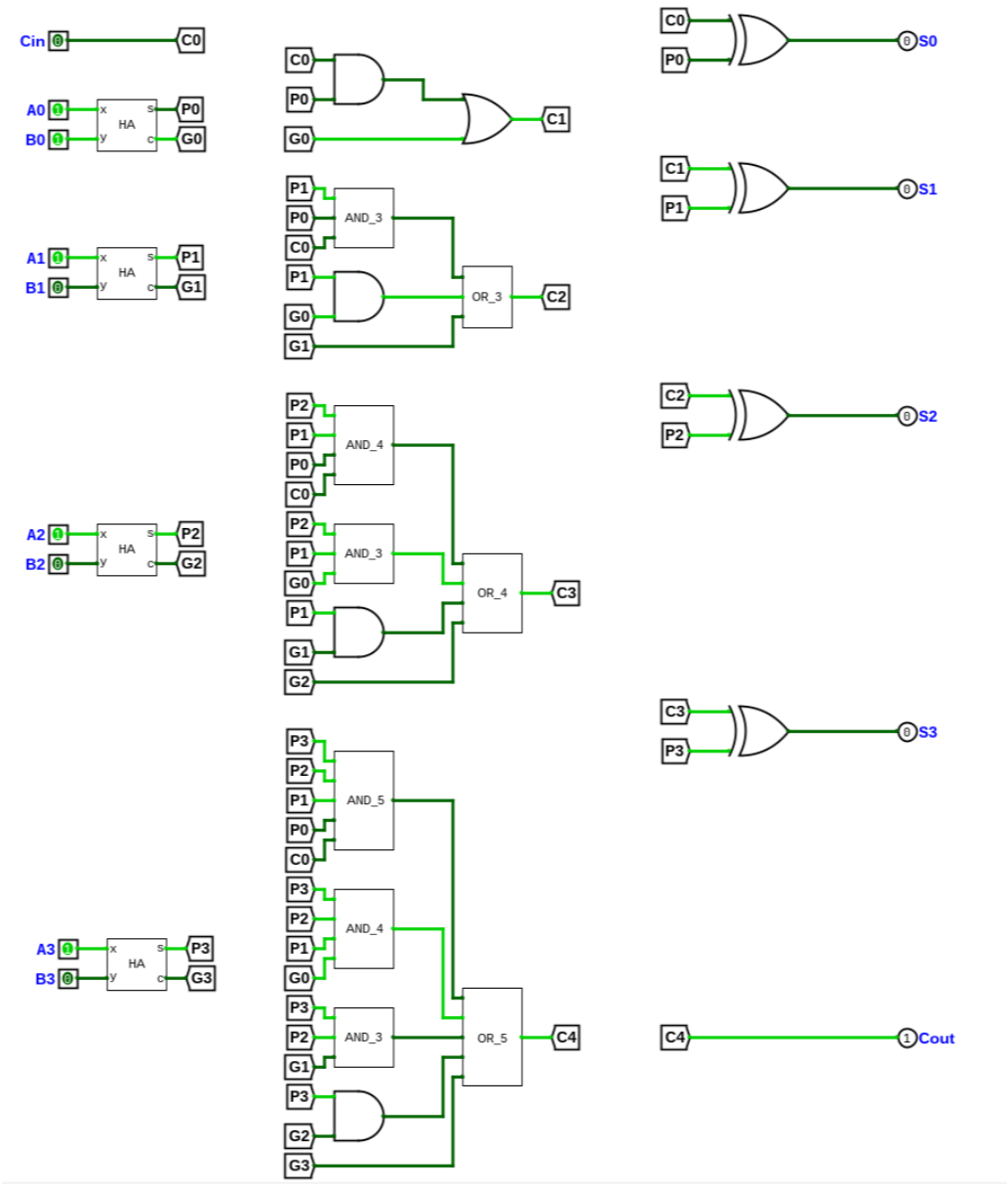

Figure 6: 4-Bit Carry Lookahead Adder implemented with Tunnels and Parallel Carry Logic.